

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice

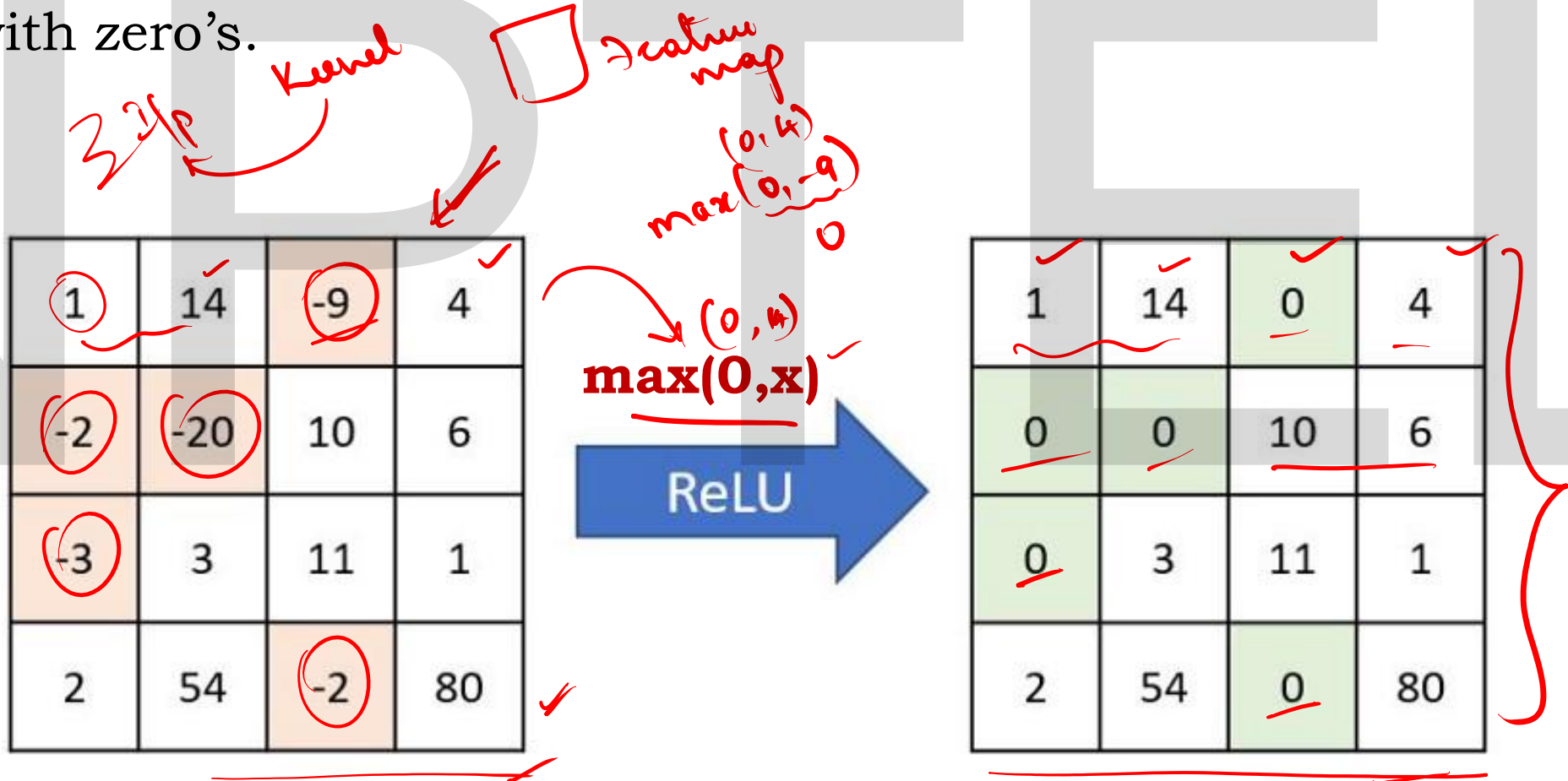
Week 1: Fundamentals of Deep Learning for Generative AI

Today's Session Overview

- Explore the next layers in a CNN:
 1. Activation Layer (ReLU)
 2. Pooling Layer
 3. Flatten Layer
 4. Fully Connected Layer
- Understand how these layers work together in a deep CNN architecture.

Activation Layer (ReLU)

In this layer we remove every negative values from the filtered images and replaces it with zero's.



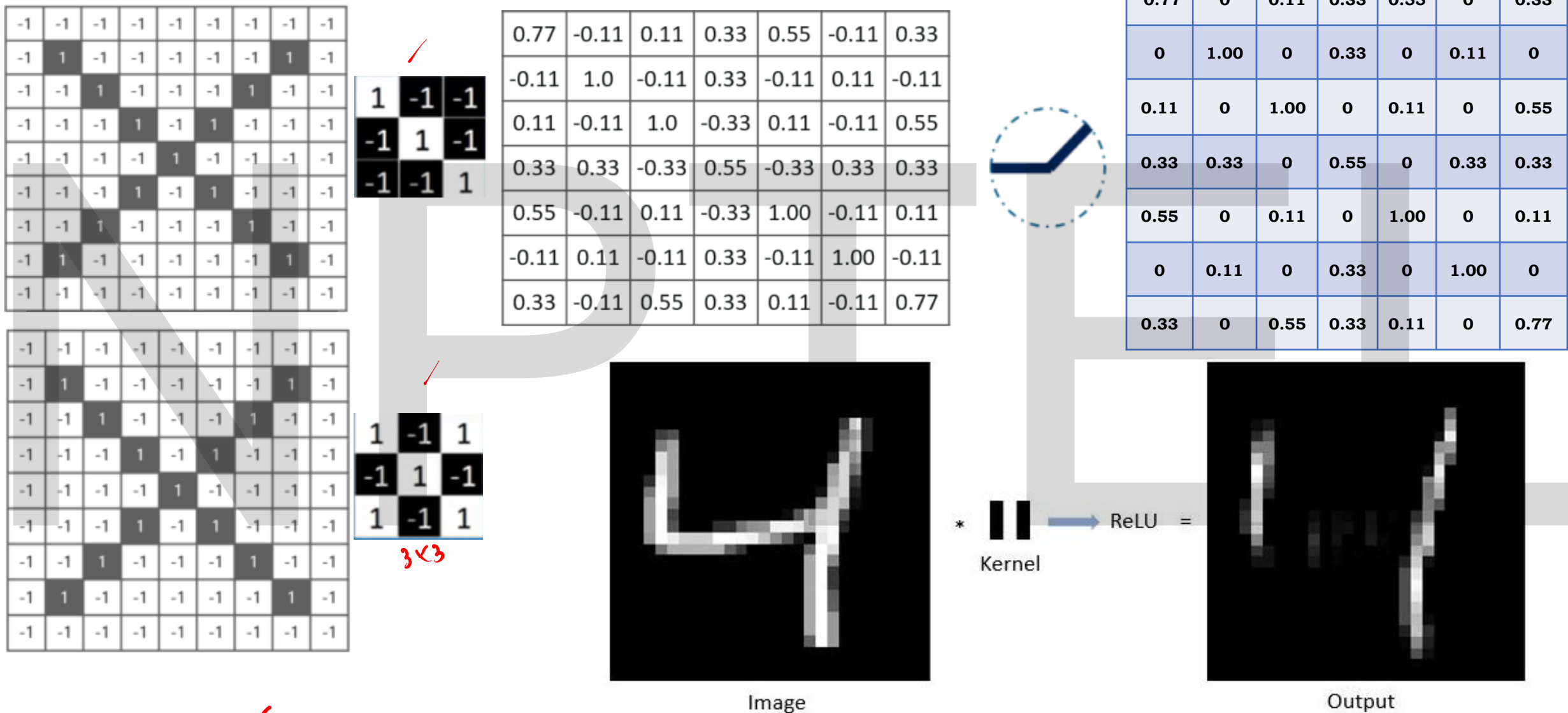
$$\max(0, x)$$

ReLU

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

ReLU



Conv2D(32,(3,3),input_shape=(64,64,1),activation='relu')

Calculating Number of Parameters

The formula is:

$$\text{Parameters} = (f_h \times f_w \times n_c + 1) \times n_f$$

Where:

$f_h \times f_w$: Kernel/Filter height and width

n_c : number of input channels

n_f : Number of filters

+1 bias term per filter

Let's say:

- Input: **32 × 32 × 3** (RGB image)
- Filter size: **5 × 5**
- Number of filters: **6**
- Stride = any value (doesn't affect parameter count)
- Padding = any value (doesn't affect parameter count)

$$\text{Parameters} = (f_h \times f_w \times n_c + 1) \times n_f$$

$$\text{Parameters} = (5 \times 5 \times 3 + 1) \times 6 = 456$$

Layer	Filter height & width	Input Channels	# of Filters	Total Parameters
Conv1	3×3	3	64	$(3 \times 3 \times 3 + 1) \times 64 = 1,792$
Conv2	5×5	64	128	$(5 \times 5 \times 64 + 1) \times 128 = 204,928$

Calculating the output when an image passes through a convolutional layer

Formula:

$$\text{Output size} = \left(\frac{I - f + 2P}{S} \right) + 1 \times D$$

Where

I : Input

f: Filter/Kernel size

S: Stride

P: Padding

D: Number of output channels (filters), which becomes the depth of the output.

Image: $32 \times 32 \times 3$
Filter size: 5×5
Number of filters: 6
Stride = 1
Padding = 0

Without padding

$$\begin{aligned} \text{Output size} &= \left(\frac{I - f + 2P}{S} \right) + 1 \times D \\ &= \left(\frac{32 - 5 + 0}{1} + 1 \right) \\ &= 28 \end{aligned}$$

Output:

$$28 \times 28 \times 6$$

Image: $32 \times 32 \times 3$
Filter size: 5×5
Number of filters: 6
Stride = 1
Padding = 2

With padding

$$\begin{aligned} \text{Output size} &= \left(\frac{I - f + 2P}{S} \right) + 1 \times D \\ &= \left(\frac{32 - 5 + 4}{1} + 1 \right) \\ &= 32 \end{aligned}$$

Output:

$$32 \times 32 \times 6$$

Pooling Layer

Pooling in CNNs is a **downsampling technique** that reduces the spatial dimensions (width and height) of feature maps, thereby decreasing computational cost.

1			

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

How It Works:

Sliding Window: A fixed-size window (e.g., 2x2) slides over the input feature map.

Summary Operation: For each window, applying the summary operation such as max pooling (taking the maximum value) or average pooling (taking the average value), to produce a smaller output.

Output Generation: The chosen value becomes a single output, creating a downsampled feature map. This process is repeated across the entire input feature map.

Types of Pooling

Stride and the window size
 Stride: 2, Window size: 2×2

8	1	6	2
71	0	6	6
50	30	7	7
20	60	13	3

Feature map of size
 4×4

71	6
60	13

Max Pooling
 2×2

20	5
40	7.5

Average Pooling

0	2
20	3

Min Pooling

71	5.5
50	10

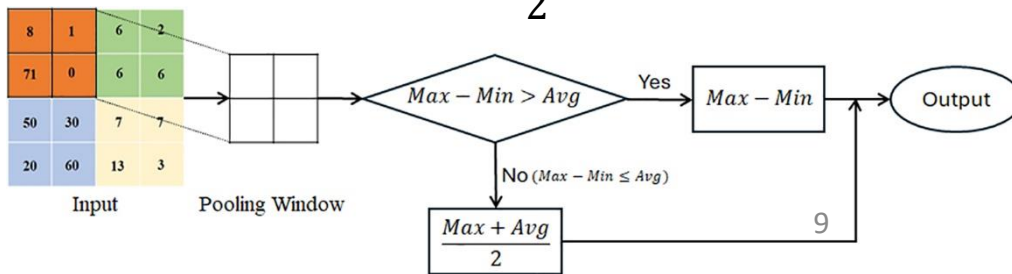
Max Avg Min Pooling

When the difference between the maximum and minimum value is large, it means the window has a strong feature or sharp change. In that case, it keeps the strong information using:

$$Max - Min$$

When the difference is small or moderate, it means the window is more uniform. In that case, it gives a smoother value using:

$$\frac{Max + Avg}{2}$$



0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

Max pooling

1	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

`MaxPooling2D(pool_size=(2,2))`

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
```

```
model = Sequential([
```

```
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1), padding = 'same'),
```

```
    MaxPooling2D(pool_size=(2,2), padding = 'same')
```

```
])
```

pool_size: size of the window (e.g., (2,2))

strides: step size; default = pool_size

`MaxPooling2D(pool_size=(2,2), stride=(1,1))`

Calculating Output size of a Max Pooling

Formula:

Output size

Where,

I : Input to 1st layer

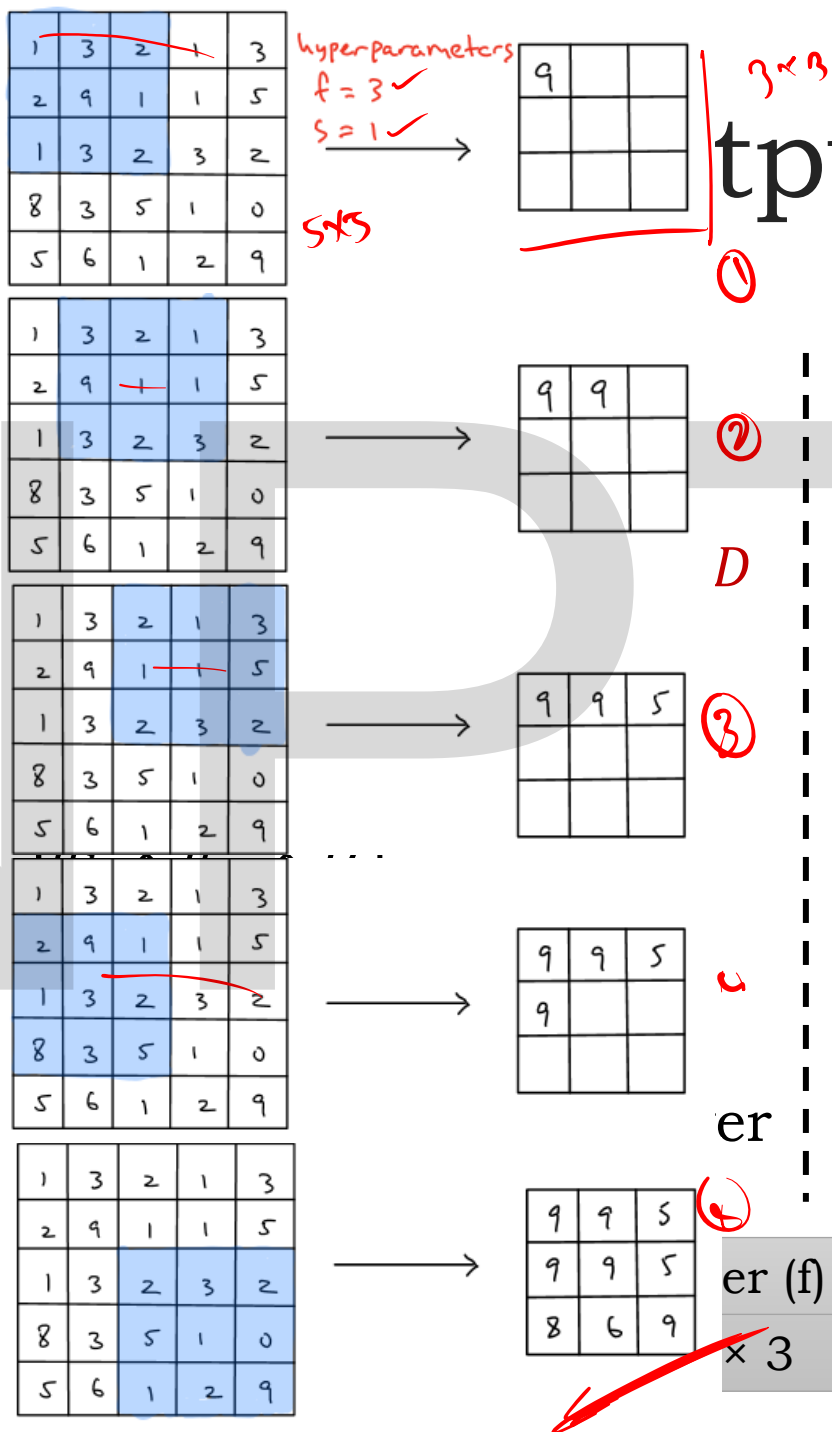
f: Filter size

S: Stride

D: Number of conv layers

D: Number of conv layers

Layer Type
Max Pooling



Output size of a Max Pooling

Let's say we have:

Input: $5 \times 5 \times 1$ (i.e., height = 5, width = 5, depth = 1)

Max Pooling:

- Filter size = 3
- Stride = 1

Apply formula to height and width:

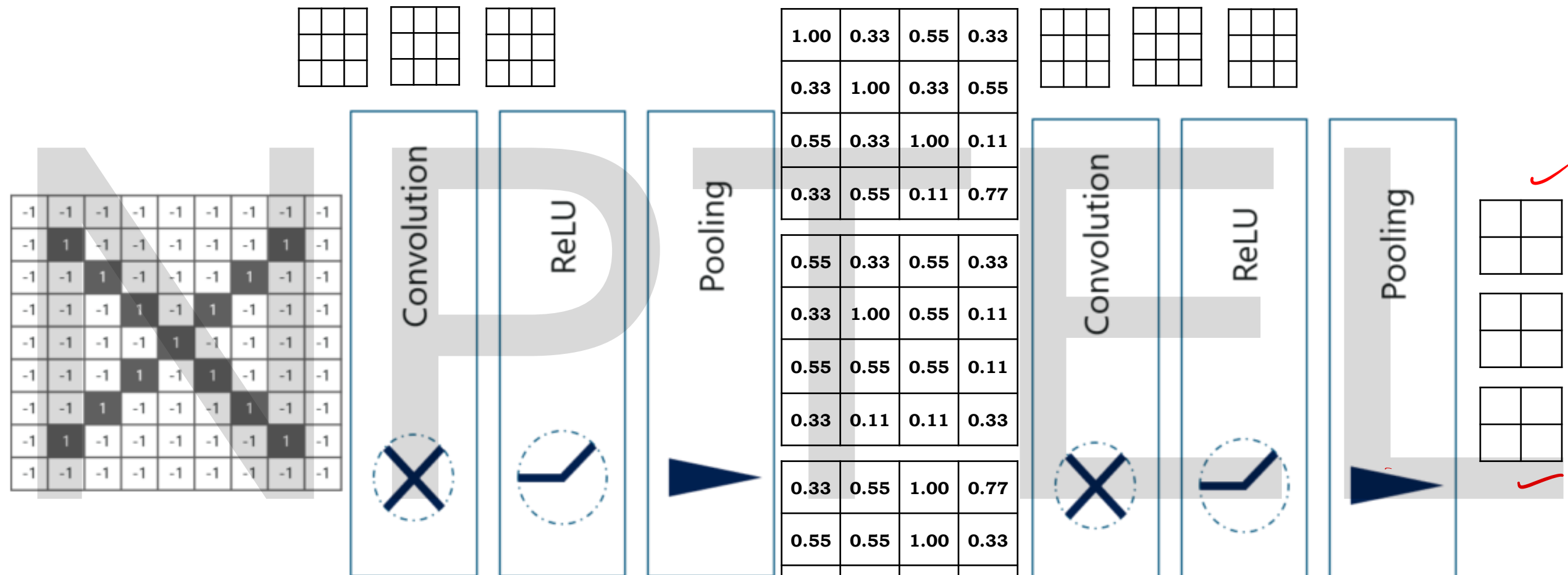
Output Height = $\left\lfloor \frac{5-3}{1} + 1 \right\rfloor = 3$

Output Width = $\left\lfloor \frac{5-3}{1} + 1 \right\rfloor = 3$

Final output = 3×3

If padding is applied during the conv layer, then the output size is $\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$

Input (f)	Stride (S)	Output Size
5×5	1	$3 \times 3 \times 1$



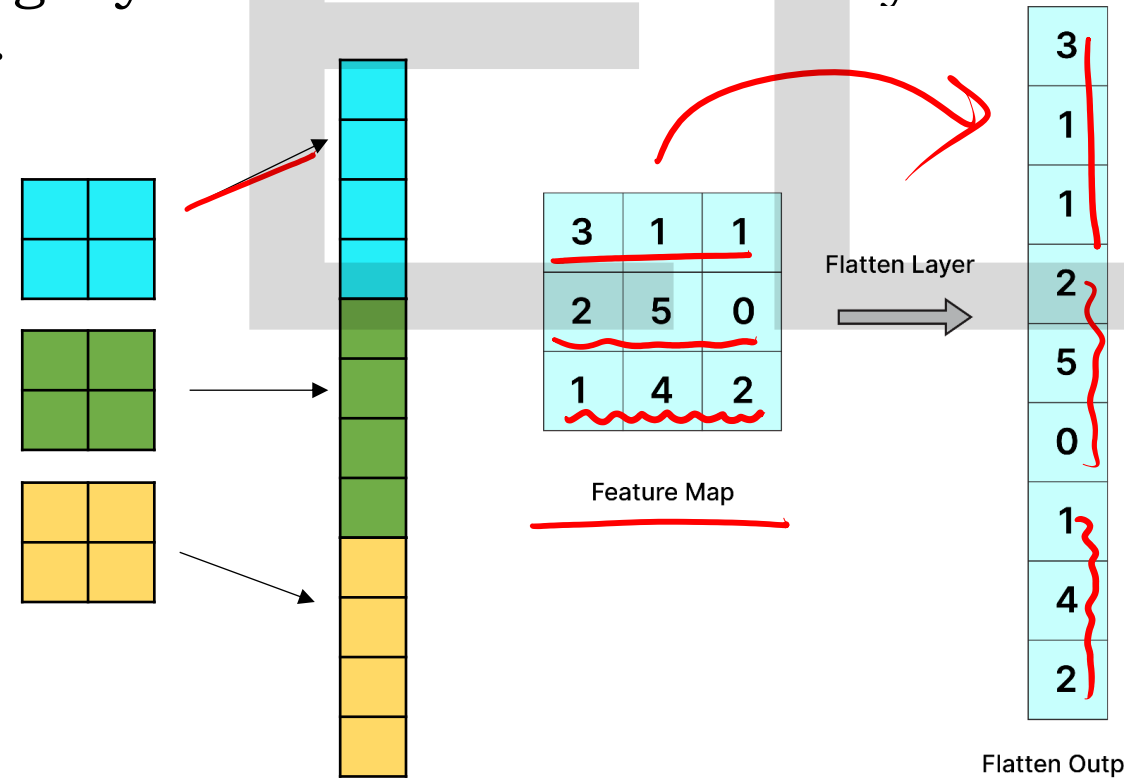
```
model = Sequential([
    Conv2D(3, (3,3), activation='relu', padding = 'same',
        input_shape=(7,7,1)),
    MaxPooling2D(pool_size=(2,2), padding='same')
])
```

```
Conv2D(3, (3,3), activation='relu',
padding = 'same'),
MaxPooling2D(pool_size=(2,2),
padding = 'same')
```

Flatten and Fully Connected layers

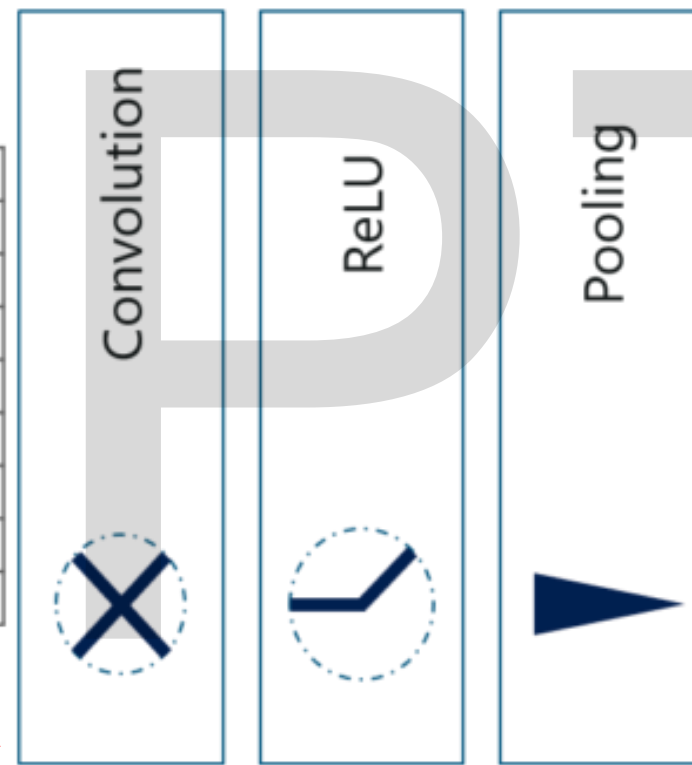
- The primary purpose of the flatten layer is to **convert the multi-dimensional output of the convolutional and pooling layers into a one-dimensional vector**. This transformation is necessary **because fully connected layers**, which are **typically used for classification, require a 1D input**.
- The flatten layer is placed after the final pooling layer and before the first fully connected (dense) layer in a CNN architecture.

`model.add(Flatten())`



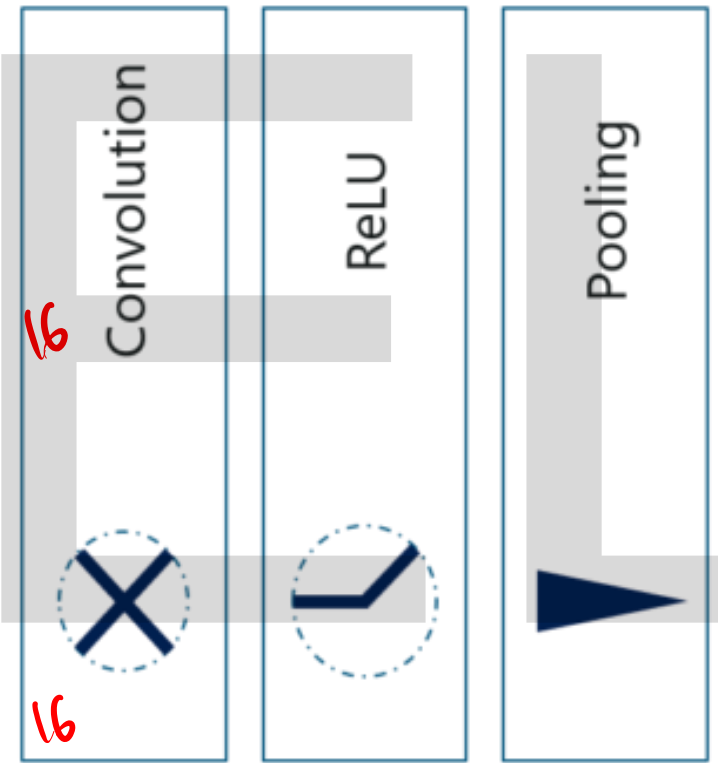
-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

7x7
256x256



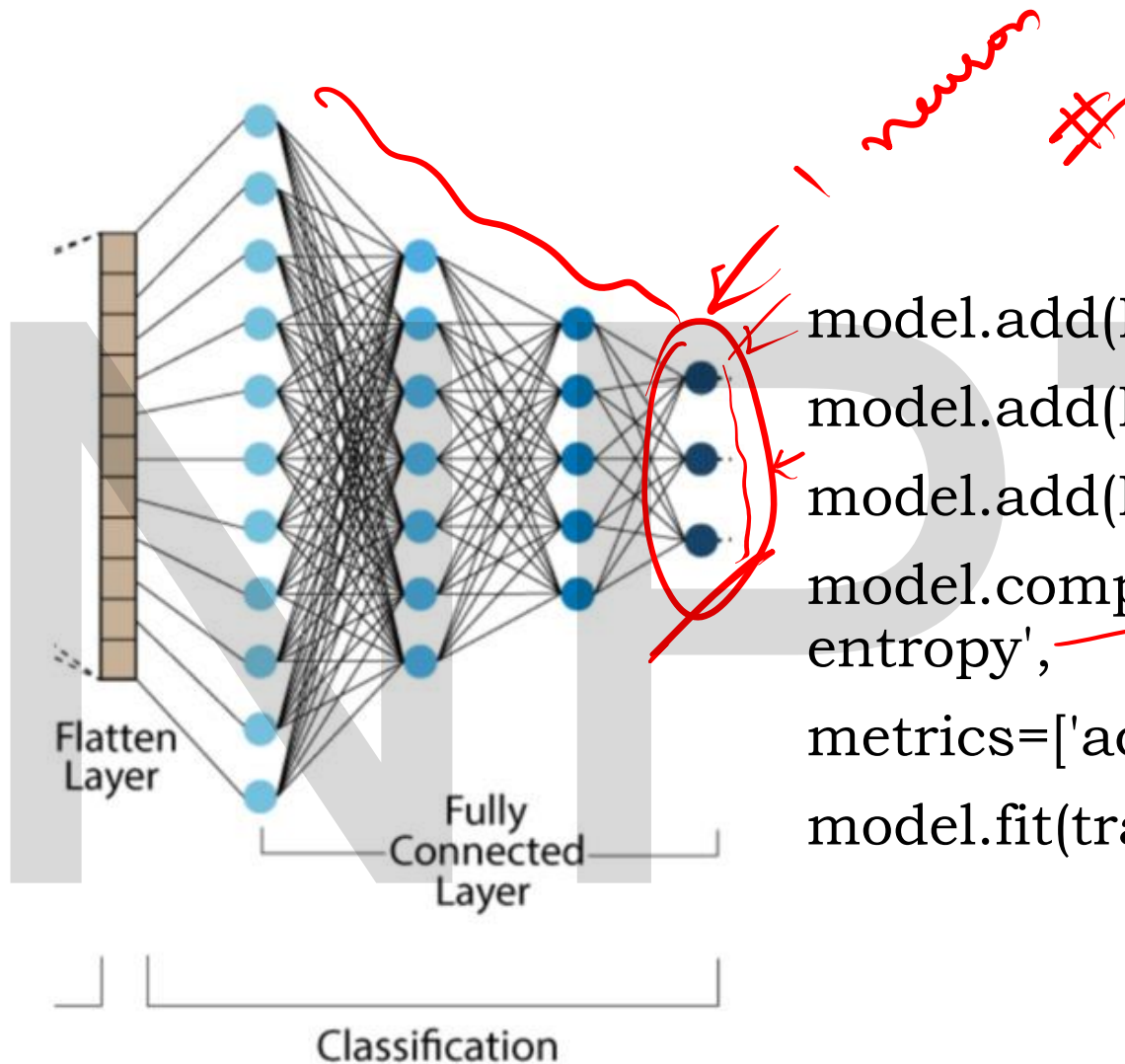
1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77
0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33
0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

16



12 neurons

48 neurons



`model.add(Flatten())` ✓

`model.add(Dense(128, activation='relu'))`

`model.add(Dense(units=3, activation='softmax'))` *sigmoid*

`model.compile(optimizer='adam', loss='categorical_crossentropy',`

`metrics=['accuracy'])` ✓

`model.fit(training_set, epochs = 10)`

Next session

Hands-on Exercise 1: Build a CNN for a real-world dataset

Hands-on Exercise 2: Building an Ensemble Model

NIPTEEL

Thank you